

# Rapport d'Analyse du Projet Invoksa

## 1. Fonctionnalités Existantes

L'application Invoksa (développée en Flutter) dispose actuellement des modules fonctionnels suivants :

- **Authentification** : Connexion et inscription par email et mot de passe.
- **Tableau de bord (Dashboard)** : Affichage des revenus globaux, et liste des factures récentes.
- **Gestion des Factures** :
  - Liste complète des factures.
  - Outils de recherche, filtrage (Payé, En attente, Non payé) et tri (Plus récent, Plus ancien, Montant élevé/faible, Nom du client).
  - Création de factures.
- **Gestion des Clients** :
  - Liste des clients.
  - Barre de recherche pour trouver un client.
  - Création de clients.
- **Paramètres** :
  - Changement de la devise et des taxes.
  - Configuration de la langue de l'application (Support FR, EN, AR).
  - Liens vers les termes et conditions & politiques de confidentialité.
  - Déconnexion.

## 2. Fonctionnalités Restant à Implémenter

Plusieurs éléments d'interface sont présents mais les fonctionnalités ne sont pas encore branchées :

- **Authentification Sociale** : La connexion via Google et Apple.
- **Récupération de compte** : Le processus de "Mot de passe oublié".
- **Notifications** : Le centre de notifications et la gestion des alertes.
- **Profil Utilisateur** :
  - Édition des informations de contact (Profil personnel).
  - Changement du mot de passe.
- **Centre d'Aide & À propos** : Écrans d'assistance et détails sur la version de l'application.
- **Tableaux dynamiques (Charts)** : Les graphiques sur le tableau de bord reposent actuellement sur des données statiques/hardcodées, ils doivent être reliés aux données réelles.

## 3. Bugs Identifiés et Problèmes de Fonctionnement

- **Données factices dans la vue** : Le graphique à barres du Dashboard utilise des tailles prédéfinies codées en dur (ex: `heights = [0.6, 0.25, ...]`) au lieu d'utiliser des statistiques réelles fournies par le `DashboardViewModel`.
- **Incohérence de navigation** : Dans `invoice_screen.dart` et `clients_screen.dart`, une icône "Paramètres" (Settings) est présente dans l'AppBar mais ne déclenche aucune action.
- **Gestion d'états vides (Empty States)** : Certaines listes vides affichent uniquement des textes bruts (ex: "Aucune facture !"), ce qui dénote par rapport à l'interface premium de l'application.

## 4. Boutons et Éléments de l'Interface Inopérants

Un certain nombre de boutons ont une action vide (`onPressed: () {}` ou `onTap: () {}`) :

- **Écran de Connexion / Inscription :**
  - Bouton "Mot de passe oublié ?"
  - Boutons de connexion sociale (Google et Apple).
- **Écran Dashboard :**
  - Icône de notifications (en haut à droite).
- **Écrans Factures & Clients :**
  - L'icône de paramètres dans la barre de titre de ces deux écrans.
- **Écran Paramètres ( `settings_screen.dart` ) :**
  - "Profil personnel"
  - "Notifications"
  - "Mot de passe"
  - "Centre d'aide"
  - "À propos"
- **Écran de détails client & Création de factures :** Des icônes et boutons d'action existent mais ne font rien.

## 5. Suggestions d'Amélioration (UX et Stabilité)

### Améliorations de l'Expérience Utilisateur (UX)

1. **Feedback visuel (Boutons désactivés / En construction) :** Pour les boutons non fonctionnels, utilisez une alerte éphémère (SnackBar ou Toast) indiquant "Fonctionnalité à venir" plutôt qu'une interaction silencieuse.
2. **Empty States Premium :** Remplacez les textes basiques par des visuels travaillés (illustrations + bouton d'action explicite "Créer votre première facture").
3. **Graphiques interactifs :** Utiliser un package comme `fl_chart` pour construire des graphiques répondant aux interactions (tooltip au survol, animations dynamiques de chargement).
4. **Cohérence de l'AppBar :** Les boutons de "Paramètres" se trouvant dans les AppBars de Factures et Clients devraient mener vers la même page Settings que celle du Dashboard, ou bien être retirés si inutilement redondants.

### Stabilité du Projet

1. **Séparation Vue / Logique :** Extraire les données statiques des widgets (comme les hauteurs du graphique). Les ViewModels devraient être l'unique source de vérité.
2. **Gestion des erreurs (Error Handling) :** Mieux isoler le `errorMessage` des ViewModels, de sorte qu'il affiche un message convivial (friendly) au lieu d'une trace d'erreur technique, le tout intégré à des modales ou BottomSheets.
3. **Optimisation des formulaires :** Assurez-vous d'utiliser `Form` et `TextFormField` avec un ensemble solide de validateurs côté front-end.
4. **Pagination :** Au fur et à mesure que les clients créés augmenteront, lister toutes les factures et tous les clients en une fois causera des lenteurs s'il n'y a pas de `ScrollController` doté d'une pagination (Infinite Scrolling).